MACHINE LEARNING INTEGRATED HEALTH ADVISORY CHATBOT SYSTEM

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

by

S. RAMA DEVI	22BFA33315
T. HEMA VARDHINI	22BFA33322
K. LEELA SIVA MARUTHI REDDY	23BFA33L12
D. NAGARJUNA	22BFA33292

Under the guidance of

Dr R. Swathi

Professor, HOD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu

Accredited by NBA, New Delhi & NAAC with 'A' grade)

Karakambadi Road, TIRUPATI – 517507

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & PermanentlyAffiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CERTIFICATE

This is to certify that the project work entitled, **"MACHINE LEARNING INTEGRATED HEALTH ADVISORY CHATBOT SYSTEM"** is a bonafide record of the project work done and submitted by

S. RAMA DEVI	**22BFA33315**
T. HEMA VARDHINI	**22BFA33322**
K. LEELA SIVA MARUTHI REDDY	**23BFA33L12**
D. NAGARJUNA	**22BFA33292**

under my guidance and supervision for the partial fulfillment of the requirements for the award of B.Tech degree in **COMPUTER SCIENCE AND ENGINEERING (AI & ML).** This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE **HEAD OF THE DEPARTMENT**

External Viva-Voce Exam held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project entitled **"Machine Learning Integrated Health Advisory Chatbot System",** done by us under the esteemed guidance of **Dr. R. Swathi,** Professor & HOD(AI&ML) and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering (AI & ML).**

This report is the result of our own effort and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

S. RAMA DEVI	**22BFA33315**
T. HEMA VARDHINI	**22BFA33322**
K. LEELA SIVA MARUTHI REDDY	**23BFA33L12**
D. NAGARJUNA	**22BFA33292**

Date:

Place:

ACKNOWLEDGEMENT

We are thankful to our guide **Dr. R. Swathi, Professor & HOD, Department of CSE (AI&ML)** for her valuable guidance and encouragement. Her helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our gratefulness and sincere thanks to **Dr. R. Swathi, Professor & HOD, Department of CSE of AI&ML,** for her kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing out hearty thanks to our beloved Principal **Dr Vijaya Gunturu** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the management for providing all the necessary facilities during course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavors.

S. RAMA DEVI **22BFA33315**

T. HEMA VARDHINI **22BFA33322**

K. LEELA SIVA MARUTHI REDDY **23BFA33L12**

D. NAGARJUNA **22BFA33292**

ABSTRACT

The Machine Learning Integrated Health Advisory Chatbot is designed to provide fast and reliable preliminary healthcare guidance based on user-reported symptoms. By leveraging advanced machine learning techniques such as XGBoost and neural networks, the system analyzes input data to predict the most probable disease and suggests appropriate medications in real time. The chatbot offers an intuitive and user-friendly interface, enabling users to easily interact and receive instant health insights. This system aims to assist individuals in early identification of potential health issues, thereby reducing delays in diagnosis and promoting timely preventive measures. Furthermore, it enhances accessibility to basic healthcare support, especially for users with limited access to medical facilities. Overall, the proposed solution demonstrates the potential of integrating machine learning with conversational interfaces to improve healthcare awareness and decision-making.

Keywords: Machine Learning, Health Advisory Chatbot, Disease Prediction, Symptom Analysis, XGBoost.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
NLP	Natural Language Processing
XGBoost	Extreme Gradient Boosting
API	Application Programming Interface
GUI	Graphical User Interface
UI	User Interface
DB	Database
MySQL	My Structured Query Language
ORM	Object Relational Mapping
JSON	JavaScript Object Notation
CSV	Comma Separated Values
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
Flask	Python Web Framework
Django	Python Web Framework
NumPy	Numerical Python
Pandas	Data Analysis Library
Scikit-learn	Machine Learning Library
TensorFlow	Deep Learning Framework
Keras	Neural Network API
CPU	Central Processing Unit

Chapter 1

INTRODUCTION

The healthcare sector plays a vital role in ensuring the well-being of individuals by providing timely diagnosis and treatment for various medical conditions. However, access to immediate medical guidance is often limited due to factors such as shortage of healthcare professionals, high consultation costs, and long waiting times in hospitals. In many cases, individuals tend to ignore early symptoms or delay seeking medical advice, which can lead to the progression of diseases and increased health risks. These challenges highlight the need for an accessible, efficient, and cost-effective solution that can provide preliminary health guidance to users.

With the rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML), there is a significant opportunity to transform traditional healthcare services. Machine learning models can analyze large volumes of medical data and identify patterns that help in predicting diseases based on symptoms. The Machine Learning Integrated Health Advisory Chatbot System is designed to utilize these technologies to provide quick and reliable health-related suggestions. By leveraging algorithms such as XGBoost and Neural Networks, the system can accurately predict the most probable disease based on user-entered symptoms.

The proposed system offers an interactive chatbot interface that enables users to input their symptoms in a simple and user-friendly manner. Once the input is received, the system processes the data and applies trained machine learning models to generate predictions. In addition to identifying potential diseases, the system also provides recommendations such as basic medicines and precautionary measures. This ensures that users receive immediate guidance, helping them take early actions before consulting a medical professional.

Furthermore, the system incorporates a medical knowledge base that stores information related to symptoms, diseases, and treatments. This enhances the accuracy and reliability of the predictions and recommendations provided. The integration of a decision support module ensures that the outputs are meaningful and tailored to user needs. By combining data-driven analysis with

a conversational interface, the system bridges the gap between technology and healthcare accessibility.

This intelligent approach addresses the limitations of traditional healthcare systems, including delays, high costs, and limited accessibility. It promotes early diagnosis, improves awareness, and empowers users to make informed health decisions. Additionally, the system reduces the burden on healthcare facilities by minimizing unnecessary visits for minor health issues.

In conclusion, the Machine Learning Integrated Health Advisory Chatbot System represents a significant step towards digital healthcare innovation. It demonstrates how AI-driven solutions can enhance healthcare accessibility, improve efficiency, and provide timely assistance to users. This project highlights the potential of integrating machine learning with user-friendly applications to create smart and scalable healthcare support systems.

.

Chapter 2

PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION:

The traditional approach to obtaining medical advice is often time-consuming, costly, and limited by the availability of healthcare professionals. Patients typically need to visit hospitals or clinics for even minor health concerns, which leads to long waiting times and increased workload for medical staff. In many cases, individuals delay seeking medical consultation due to inconvenience or lack of access, especially in rural or remote areas where healthcare facilities are scarce. This delay can result in the worsening of health conditions and reduced chances of effective treatment.

Moreover, early-stage symptoms of many diseases are often ignored or misunderstood by individuals due to a lack of medical awareness. People frequently rely on self-diagnosis through unreliable online sources, which may lead to incorrect conclusions and inappropriate treatments. The absence of a reliable and easily accessible system for preliminary health assessment increases the risk of misdiagnosis and complications. Additionally, traditional systems do not provide instant feedback or guidance, making it difficult for users to take timely preventive measures.

With the rapid advancement of technology, there is a growing demand for intelligent systems that can provide quick and accurate health-related suggestions. However, existing healthcare applications often lack proper integration of machine learning models or fail to deliver personalized and real-time responses. Many systems are limited to basic symptom checking without offering meaningful recommendations such as medicines or precautions. Furthermore, the lack of a structured medical knowledge base and decision support mechanisms reduces the effectiveness of such systems.

Another major challenge is ensuring consistency and accuracy in disease prediction. Human-based diagnosis can sometimes vary depending on experience and judgment, whereas automated systems require well-trained models and reliable datasets to produce consistent outputs. Therefore, there is a clear need for a scalable, efficient, and user-friendly solution that can analyze symptoms, predict diseases, and provide appropriate health guidance in real time.

2.2 PROJECT DETAILS

This project, titled *Machine Learning Integrated Health Advisory Chatbot System*, aims to develop an intelligent system that provides preliminary health guidance based on user-input symptoms. The system utilizes machine learning techniques, particularly XGBoost and Neural Network models, to analyze symptoms and predict the most probable disease. These models are trained on structured medical datasets containing various symptoms and their corresponding diseases, enabling the system to make accurate predictions.

The process begins with user interaction through a chatbot interface available on web or mobile platforms. Users can enter their symptoms in a simple and user-friendly manner. The input data is then processed by the symptom handler module, which validates and formats the data into a structured form suitable for machine learning analysis. This preprocessing step ensures that the input is clean and consistent, improving the accuracy of predictions.

Once the data is processed, it is passed to the prediction module, where trained machine learning models analyze the symptoms and generate disease predictions. The system identifies the most likely disease based on learned patterns from the dataset. In addition to prediction, the system is integrated with a medical knowledge base that stores information about diseases, symptoms, medicines, and precautionary measures. This knowledge base plays a crucial role in supporting both prediction and decision-making processes.

After the disease is predicted, the decision support module maps the results to appropriate medicines and preventive measures. This ensures that users receive not only the predicted disease but also actionable recommendations for managing their condition. The response generator module then converts the output into a clear and user-friendly format, which is displayed to the user through the chatbot interface.

The system also includes a storage mechanism to maintain user queries and prediction results for future reference and analysis. Performance evaluation of the models is carried out using metrics such as accuracy, precision, recall, and F1-score. The overall design ensures real-time response, ease of use, and scalability, making it suitable for practical implementation in healthcare applications.

2.3 SCOPE OF THE PROJECT

The project titled *Machine Learning Integrated Health Advisory Chatbot System* focuses on developing an intelligent and accessible platform for providing preliminary healthcare guidance. The system aims to bridge the gap between users and healthcare services by offering quick and reliable disease prediction based on symptoms. By leveraging machine learning techniques, the project enhances the accuracy and efficiency of health advisory systems compared to traditional methods.

The system allows users to interact through a chatbot interface and input their symptoms conveniently. It processes the data, predicts possible diseases, and provides recommendations such as medicines and precautions. This functionality makes the system particularly useful for individuals seeking immediate guidance for minor health issues. It can be used by a wide range of users, including students, working professionals, and people in remote areas with limited access to healthcare facilities.

The technological framework of the project includes machine learning libraries such as Scikit-learn and TensorFlow for model development. The system can be integrated into web and mobile applications, ensuring accessibility and ease of use. Additionally, the project supports scalability, allowing future enhancements such as real-time data integration, multi-language support, and advanced predictive models.

Overall, the scope of this project lies in providing a user-friendly, efficient, and scalable healthcare solution that leverages artificial intelligence to improve early diagnosis and health awareness**.**

Chapter 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Current healthcare advisory systems largely depend on direct consultation with medical professionals or basic symptom-checking applications. Traditional methods require patients to visit hospitals or clinics, where doctors analyze symptoms and provide diagnosis and treatment. While effective, this process is time-consuming, costly, and often inaccessible to individuals in remote or underserved areas. The dependency on physical consultations leads to delays in receiving medical guidance, especially for minor health issues.

Some digital health platforms and symptom checker tools are available, but many of them provide only generic information without accurate disease prediction. These systems often rely on static rules or predefined mappings between symptoms and diseases, which limits their effectiveness. They do not utilize advanced machine learning techniques, resulting in less accurate and non-personalized outputs. Moreover, many existing systems fail to provide comprehensive recommendations such as medicines and precautionary measures.

Another limitation of current systems is the lack of real-time response and interactivity. Users often receive delayed or incomplete guidance, which may not be sufficient for early decision-making. Additionally, there is limited integration of medical datasets and predictive models, reducing the reliability of the system. As a result, users may still depend on unreliable online sources or self-diagnosis, which can lead to incorrect conclusions and health risks.

3.1.1 Disadvantages

- ***Time-Consuming Process:***

  Traditional consultations require time and physical visits.

- ***Limited Accessibility:***

  Healthcare services are not easily accessible in remote areas.

- ***Lack of Personalization:***

  Generic symptom checkers do not provide accurate predictions.

- ***No Real-Time Guidance:***

  Delayed responses reduce effectiveness.

- ***Human Dependency:***

  Heavy reliance on doctors for minor health issues.

- ***Inconsistent Information:***

  Different sources provide varying medical advice.

- ***No Intelligent Prediction:***

  Lack of machine learning integration limits accuracy.

3.2 PROPOSED SYSTEM

The proposed system introduces a Machine Learning Integrated Health Advisory Chatbot System designed to provide quick and reliable health guidance. The system utilizes advanced machine learning models such as XGBoost and Neural Networks to analyze user-entered symptoms and predict the most probable disease. This approach ensures higher accuracy and consistency compared to traditional methods.

The process begins with user interaction through a chatbot interface available on web or mobile platforms. Users enter their symptoms, which are then processed by the symptom handler module to ensure data validity and proper formatting. The processed data is passed to the prediction module, where trained models analyze the symptoms and generate disease predictions based on learned patterns.

The system is integrated with a medical knowledge base that stores detailed information about symptoms, diseases, medicines, and precautions. After prediction, the decision support module maps the results to appropriate recommendations, including suggested medicines and preventive measures. The response generator then presents the output in a clear and user-friendly format through the chatbot interface.

This system provides real-time responses, improves accessibility, and reduces dependency on physical consultations for minor health issues. It offers a scalable and efficient solution that can be integrated into modern healthcare platforms, ensuring better user experience and early health awareness.

3.2.1 Advantages

- ***High Accuracy:***

  Machine learning models improve prediction reliability.

- ***Real-Time Response:***

  Instant health guidance is provided to users.

- ***User-Friendly Interface:***

  Chatbot ensures easy interaction.

- ***Reduced Human Dependency:***

  Minimizes need for immediate doctor consultation.

- ***Accessible Anywhere:***

  Can be used anytime via web or mobile.

- ***Data-Driven Decisions:***

  Uses trained models and medical datasets.

- ***Scalable System:***

  Can be expanded with more features and data.

- ***Improved Awareness:***

  Encourages early diagnosis and preventive care.

Chapter 4

ENVIRONMENTS

4.1 SOFTWARE REQUIREMENTS

- Operating System: Windows 10 / Windows 11
- Coding Language: Python
- Front-End: HTML, CSS, JavaScript
- Back-End: Python (Flask / Django)
- Designing: HTML, CSS
- Database: MySQL / SQLite
- Machine Learning Models: XGBoost, Neural Networks
- Libraries/Frameworks: Scikit-learn, TensorFlow / Keras, Pandas, NumPy

4.2 HARDWARE REQUIREMENTS

- Processor : I5/Intel processor
- RAM : 8 GB
- Hard Disk : 160 GB
- Key Board : Standard Windows Keyboard
- Mouse : Two or Three Button Mouse
- Monitor : SVGA

4.3 SOFTWARE FEATURES AND DEPENDENCIES

When it comes to Python-based development for machine learning applications, several libraries and frameworks provide the essential tools required for building, training, and deploying intelligent systems. Python has become the most preferred programming language in the field of data science and artificial intelligence due to its simplicity, flexibility, and extensive library support. In this project, multiple Python libraries are used to implement machine learning models, process data, and develop the chatbot system efficiently. Below are some of the most widely used libraries and frameworks in this system.

Scikit-learn

Scikit-learn is one of the most widely used libraries for implementing traditional machine learning algorithms. It provides simple and efficient tools for data mining and data analysis. In this project, Scikit-learn is used for preprocessing data, training models, and evaluating performance. It supports various algorithms that help in disease prediction based on symptoms.

Key features include:

• Supports classification, regression, clustering, and dimensionality reduction techniques.

• Provides model evaluation tools such as accuracy, precision, recall, and F1-score.

• Easy integration with libraries like NumPy and Pandas for data handling.

TensorFlow / Keras

TensorFlow, along with its high-level API Keras, is used for building and training deep learning models such as neural networks. It is an open-source framework developed by Google and is widely used for creating intelligent systems. In this project, it is used to implement neural network models for accurate disease prediction.

Key features include:

• Supports deep learning models such as Artificial Neural Networks (ANN).

• Enables fast computation using CPU and GPU.

• Provides flexibility for designing complex model architectures.

• **XGBoost**

XGBoost (Extreme Gradient Boosting) is a powerful machine learning algorithm known for its high performance and accuracy. It is used in this project for predicting diseases based on input symptoms. XGBoost works efficiently with structured data and helps improve prediction results.

Key features include:

• High accuracy and performance compared to traditional algorithms.

• Handles missing data effectively.

• Supports parallel processing for faster computation.

Pandas

Pandas is a data manipulation and analysis library used for handling structured datasets. It is used to read, clean, and organize medical data before feeding it into machine learning models.

Key features include:

• Provides DataFrame structure for easy data handling.

• Supports data cleaning, filtering, and transformation.

• Efficient handling of large datasets.

NumPy

NumPy is a fundamental library for numerical computing in Python. It is used for performing mathematical operations on arrays and matrices, which are essential for machine learning algorithms.

Key features include:

• Supports fast numerical computations.

• Provides multi-dimensional array structures.

• Works seamlessly with other ML libraries.

Flask / Django

Flask or Django is used as the backend framework for developing the chatbot system. It handles user requests, processes data, and connects the frontend with machine learning models.

Key features include:

• Lightweight and easy-to-use web framework.

• Supports REST API development.

• Enables integration of ML models into web applications.

MySQL / SQLite

The database is used to store user inputs, prediction results, and system data. It helps in maintaining records and improving system performance over time.

Key features include:

• Efficient data storage and retrieval.

• Supports structured data management.

• Ensures data consistency and reliability.

Chapter 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

System design refers to the structured planning, coordination, and integration of various modules to achieve the intended functionality of a software system efficiently. For the project titled *"Machine Learning Integrated Health Advisory Chatbot System,"* the system design outlines how individual components—including user interaction, symptom processing, machine learning-based disease prediction, decision support, and response generation—work together to deliver an intelligent and real-time healthcare advisory solution.

The system is architected as a modular pipeline consisting of six core components: chatbot interface, symptom handler, machine learning prediction engine, medical knowledge base, decision support module, and response generator. Each module is developed independently but interconnected to ensure smooth communication, efficient data flow, and real-time responsiveness throughout the system lifecycle.

The process begins with user interaction through the chatbot interface, where users enter their symptoms using a web or mobile-based platform. The chatbot interface serves as the primary communication layer between the user and the system, providing a simple and intuitive environment for input and output. The entered symptoms are then forwarded to the symptom handler module for further processing.

The symptom handler module is responsible for validating and formatting the input data. It ensures that the symptoms provided by the user are clean, structured, and suitable for machine learning analysis. This module performs preprocessing tasks such as data normalization, filtering invalid inputs, and converting textual input into a format understandable by the prediction models. Proper preprocessing improves the accuracy and reliability of the overall system.

At the core of the system lies the machine learning prediction engine, which utilizes advanced algorithms such as XGBoost and Neural Networks to analyze the processed symptom data. These models are trained on a comprehensive medical dataset containing relationships between symptoms and diseases. By learning patterns from historical data, the models can

accurately predict the most probable disease. The use of multiple models enhances prediction performance and ensures robustness across different input conditions.

The medical knowledge base plays a crucial role in supporting the prediction and recommendation process. It contains detailed information about diseases, symptoms, medicines, and precautionary measures. The prediction engine interacts with the knowledge base to retrieve relevant data, ensuring that the results are aligned with medical knowledge and real-world scenarios.

Once the disease prediction is generated, the system moves to the decision support module. This module interprets the prediction results and maps them to appropriate recommendations, including suggested medicines and preventive measures. It ensures that the system provides actionable insights rather than just raw predictions, thereby improving user understanding and decision-making.

The final stage of the system is the response generator module, which converts the processed output into a clear, concise, and user-friendly response. The chatbot presents the results to the user in an easily understandable format, completing the interaction cycle. This module enhances user experience by delivering meaningful and well-structured responses.

The entire system is designed for real-time operation, ensuring quick responses and minimal delay. The modular architecture supports scalability and future enhancements, such as integration with wearable devices, voice-based interaction, and real-time health monitoring systems. It can be deployed across web and mobile platforms, making it accessible to a wide range of users. It ensures that the system provides actionable insights rather than just raw predictions, thereby improving user understanding and decision-making.

In conclusion, the system architecture ensures efficient data processing, modular design, high accuracy, and real-time performance. It provides a strong foundation for developing an intelligent healthcare chatbot system that improves accessibility, supports early diagnosis, and enhances overall user experience in digital healthcare environments.

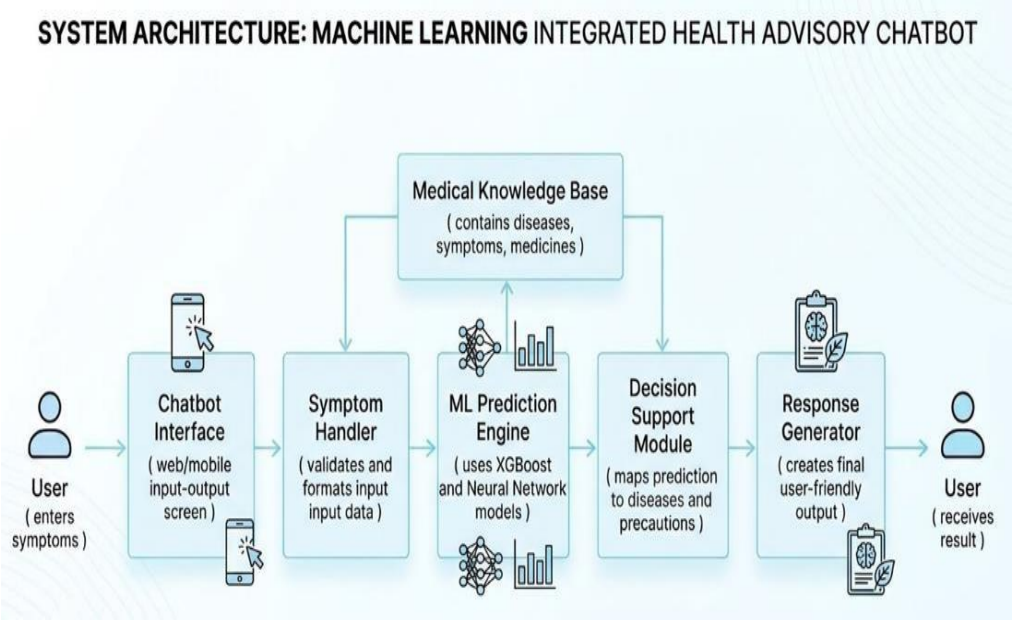

Fig 5.1: System architecture

5.2 UML DIAGRAMS

What is UML?

The Unified Modelling Language (UML) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

As the strategic value of software increases for many companies, the industry looks for techniques to automate the production of software and to improve quality and reduce cost and time-to-market. These techniques include component technology, visual programming, patterns and frameworks. Businesses also seek techniques to manage the complexity of systems as they increase in scope and scale. In particular, they recognize the need to solve recurring architectural problems,

such as physical distribution, concurrency, replication, security, load balancing and fault tolerance. Additionally, the development for the World Wide Web, while making some things simpler, has exacerbated these architectural problems. The Unified Modelling Language (UML) was designed to respond to these needs. Simply, systems design refers to the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements, which can be done easily through UML diagrams.

Contents of UML:

In general, a UML diagram consists of the following features:

- ***Entities***

  These may be classes, objects, users, or systems behaviors. Relationship Lines that model the relationships between entities in the system.

- ***Generalization***

  A solid line with an arrow that points to a higher abstraction of the present item.

- ***Association***

  A solid line that represents that one entity uses another entity as part of its behaviour.

- ***Dependency***

  A dotted line with an arrowhead that shows one entity depends on the behaviour of another.

Advantages:

- Provides standard for software development.
- Development time is reduced.
- The past faced issues by the developers are no longer exists.
- Has large visual elements to construct and easy to follow.
- To establish an explicit coupling between concepts and executable code.
- To creating a modeling language usable by both humans and machines UML defines several models for representing systems.

In this project six basic UML diagrams have been explained:

1. Class Diagram

2. Use Case Diagram
3. Sequence Diagram
4. Activity Diagram
5. Component Diagram
6. Deployment Diagram
7. State Diagram

5.2.1 Class Diagram

UML class diagrams represent the static structure of a system by illustrating the relationships between different classes that form the core architecture of the *Machine Learning Integrated Health Advisory Chatbot System*. These diagrams describe how classes are organized, their attributes, methods, and how they interact with each other. Unlike object diagrams, class diagrams provide a generalized view that applies to all objects in the system. They serve as a blueprint for implementation and help developers understand the system design at a high level.

The **User class** acts as the primary actor interacting with the system. It contains attributes such as userId, name, age, medical History, and current Symptoms. It also includes methods like login(), provideSymptoms(), viewDiagnosis(), and getRecommendations(), which allow the user to communicate with the chatbot and receive results.

The **Chatbot Interface class** manages the interaction between the user and the system. It maintains attributes like active Session and interface Type, and provides methods such as displayWelcomeMessage(), receiveUserSymptoms(), displayPredictedDiseases(), and displayHealthcareResponse(). This class ensures smooth communication and presentation of results to the user.

The **Symptom Handler class** is responsible for processing the input provided by the user. It captures raw input, normalizes symptoms, and maps them to standardized medical terms. It includes attributes like raw Input symptom and processed Symptom List, along with methods captureInputString(), normalizeSymptoms(), and mapToStandardTerms(). This module ensures that the input data is clean and structured before passing it to the prediction model.

The **Prediction Model (Abstract) class** forms the core of the system.

The **Knowledge Base class** stores medical information such as diseases and symptoms. It contains attributes like disease Count and symptom Count, and methods such as queryDiseasesBySymptom(), getDiseaseDetails(), and updateMedicalData(). This class plays a vital role in providing accurate and relevant medical data to the system.

The **Decision Support class** processes prediction results and generates meaningful healthcare recommendations. It works with prediction data and includes methods such as consolidateModelPredictions(), combineKBWithPredictions(), filterDiagnosis(), and generateHealthcareRecommendation(). It ensures that the system provides actionable insights rather than raw predictions.

The **Response Generator class** converts system output into a user-friendly format. It includes attributes like language and communicationStyle, and methods such as translateCarePlan(), generateUserFriendlyResponse(), and generateFollowUpQuestions(). This class enhances user experience by presenting clear and understandable results. The **Database class** is responsible for storing user information, interaction history, and system data. It includes attributes like patientTable and encounterLogTable, along with methods such as saveUserProfile(), loadPatientProfile(), and logInteraction().

Machine Learning Integrated Health Advisory Chatbot System. These diagrams describe how classes are organized, their attributes, methods, and how they interact with each other. Unlike object diagrams, class diagrams provide a generalized view that applies to all objects in the system. They serve as a blueprint for implementation and help developers understand the system design at a high level.

Additionally, **DTO (Data Transfer Object) classes** such as Symptom, Disease, Prediction, and Care Plan are used to transfer structured data between modules. These classes ensure efficient communication between system components. The relationships between these classes include **association, dependency, and inheritance**. The Prediction Model class uses inheritance for extending different models, while the Symptom Handler and Decision Support modules depend on the Knowledge Base for data. The Response Generator depends on the Decision Support module for generating output.

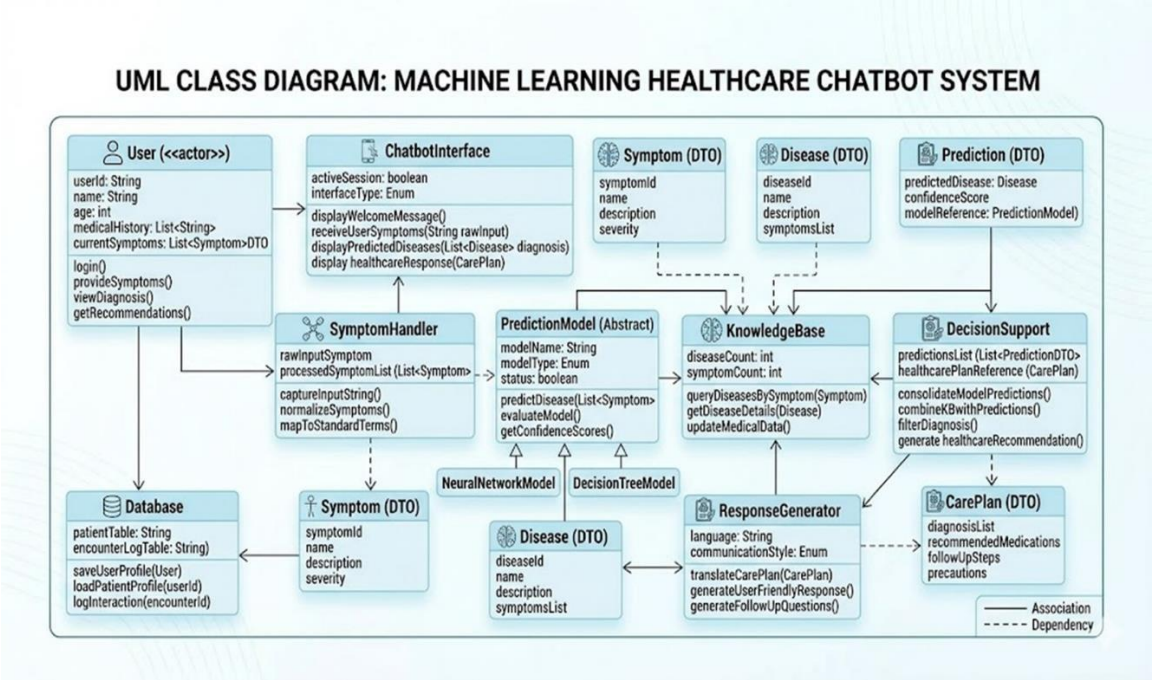

Fig 5.2.1: Class diagram

5.2.2 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a behavioral diagram that represents the functional requirements of a system by illustrating the interactions between users (actors) and the system. It provides a high-level overview of how the system operates from the user's perspective. In the *Machine Learning Integrated Health Advisory Chatbot System*, the use case diagram demonstrates how the user interacts with the chatbot to input symptoms and receive health-related predictions and recommendations.

The primary actor in this system is the **User**, who interacts directly with the system. The system boundary is represented by a rectangular box labeled *Health Advisory Chatbot System*, which encloses all the functionalities provided by the system. The use cases are represented by oval shapes, each indicating a specific function of the system.

The main use case begins with **Enter Symptoms**, where the user inputs health-related symptoms such as fever, headache, or cough into the chatbot interface. This is the starting point of the system interaction and is essential for initiating the prediction process. The use cases are represented by oval shapes, each indicating a specific function of the system.

Once the symptoms are entered, the system performs **Validate Input**, which ensures that the provided data is correct, complete, and in an appropriate format. This step is important to maintain the accuracy and reliability of the system, as improper input may lead to incorrect predictions.

After validation, the system proceeds to the **Predict Disease** use case. In this stage, the machine learning models analyze the processed input and predict the most probable disease based on learned patterns from the dataset. This is the core functionality of the system. The diagram also shows that the **Predict Disease** use case has an «include» relationship with other use cases such as **View Prediction** and **Get Medicines**. The «include» relationship indicates that these functionalities are essential parts of the prediction process.

The **View Prediction** use case allows the user to see the predicted disease results generated by the system. It ensures that users receive clear information about their health condition. The **Get Medicines** use case provides the user with recommended medicines and precautionary measures based on the predicted disease. This enhances the usefulness of the system by offering actionable health advice. The relationships between the actor and use cases are represented by association lines, indicating how the user interacts with different functionalities

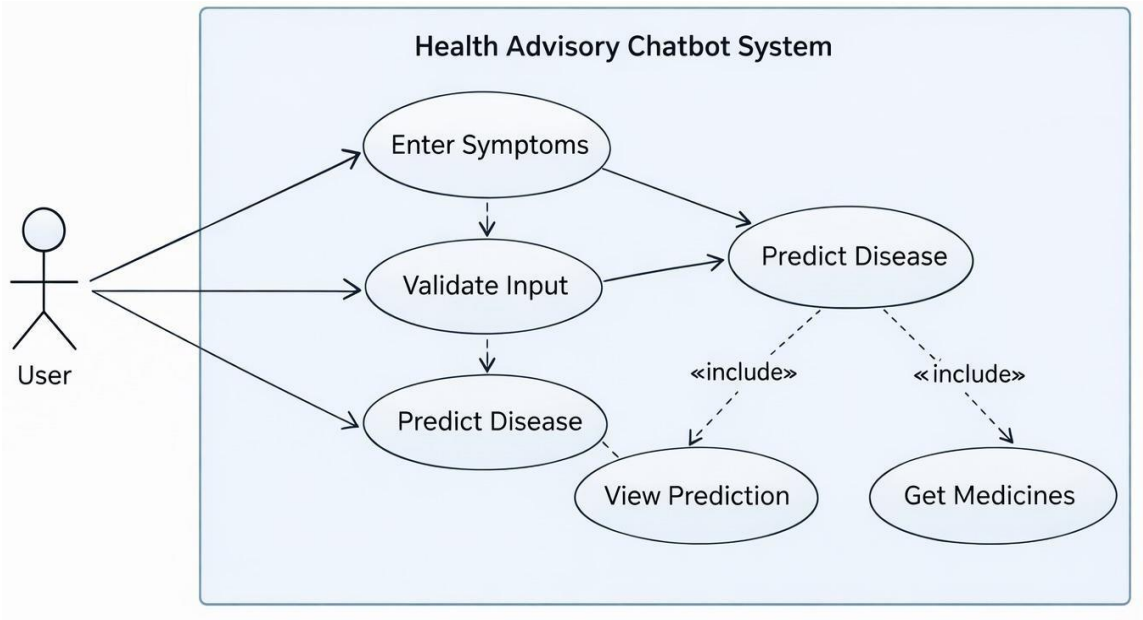

Fig 5.2.2: Use Case diagram

5.2.3 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a type of interaction diagram that illustrates how different components of a system communicate with each other and in what sequence. It represents the dynamic behavior of the system by showing the flow of messages between objects over time. The sequence diagram is particularly useful for understanding how the *Machine Learning Integrated Health Advisory Chatbot System* processes user input and generates responses step by step.

In this project, the sequence diagram describes the interaction between key components such as **User, Chatbot Interface, Symptom Handler, Prediction Module, Decision Support Module, and Response Generator**. These components work together in a sequential manner to process user symptoms and provide appropriate health recommendations.

The sequence begins with the **User**, who initiates the interaction by entering symptoms such as fever, cough, or headache. This input is sent to the **Chatbot Interface** in the form of a query. The chatbot interface acts as the communication layer between the user and the backend system.

The Chatbot Interface then forwards the raw input to the **Symptom Handler** module. The Symptom Handler processes the input by parsing and extracting relevant symptom data. This step ensures that the input is converted into a structured format suitable for further processing. Once the symptoms are processed, the data is passed to the **Prediction Module**, which is responsible for predicting the disease. The module uses machine learning models to analyze the extracted symptoms and generate a diagnosis. This is a critical step where the system determines the most probable disease based on the given input.

After generating the prediction, the Prediction Module sends the results to the **Decision Support Module**. This module interprets the predicted diagnosis and generates a care plan, which may include recommended medicines, precautions, and health advice. In some cases, an optional response or follow-up interaction may also occur, depending on user requirements

The Decision Support Module then passes the processed information to the **Response Generator**, which creates a user-friendly response. This response includes the predicted disease and suggested recommendations in a clear and understandable format.

Finally, the Response Generator sends the final output back to the Chatbot Interface, which displays the response to the user. In some cases, an optional response or follow-up interaction may also occur, depending on user requirements.

Elements of Sequence Diagram

• **Objects**

Objects represent the different components involved in the system, such as User, Chatbot Interface, Symptom Handler, Prediction Module, Decision Support, and Response Generator.

• **Lifeline**

A lifeline is represented by a vertical dashed line that shows the existence of an object during the interaction. Each component in the system has its own lifeline.

• **Messages**

Messages are shown as arrows between objects, representing communication.

• **Activation Bars**

These indicate the time during which an object is performing an action or processing a request

Steps to Create Sequence Diagram

• Identify the interacting components (User and system modules)

• Define the sequence of messages between components

• Arrange components in order from left to right

• Draw lifelines and message arrows

• Represent processing using activation bars

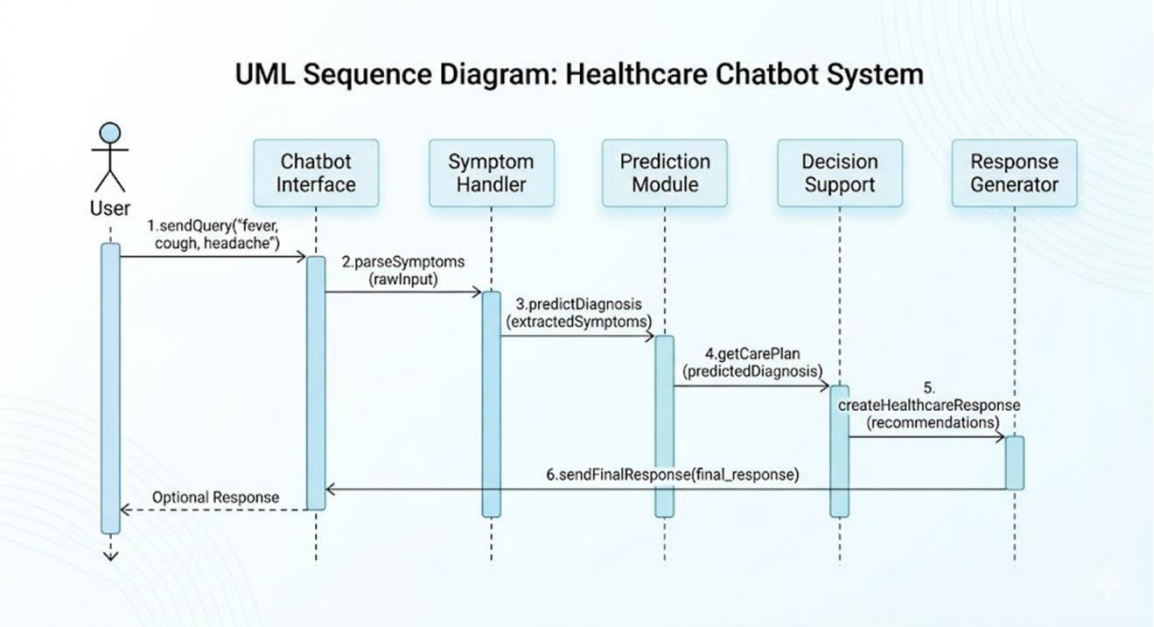

Fig 5.2.3: Sequence diagram

5.2.4 Activity Diagram

An activity diagram in Unified Modeling Language (UML) is used to represent the dynamic workflow of a system. It illustrates the sequence of activities and actions performed, along with the decision points and flow of control from one activity to another. The activity diagram provides a clear understanding of how the *Machine Learning Integrated Health Advisory Chatbot System* operates step by step, from user input to final output generation.

In this project, the activity diagram describes the complete flow of operations starting from the initial state to the final result display. It highlights how user input is processed, analyzed, and transformed into meaningful health advice through various system modules.

The process begins with the **initial state**, represented by a filled circle, indicating the start of the system. The first activity is **Enter Symptoms**, where the user provides health-related inputs such as fever, cough, or headache through the chatbot interface.

After entering symptoms, the system performs **Process Input**, where the input data is validated and formatted. At this stage, the system checks whether the provided symptoms are sufficient for analysis. If the symptoms are insufficient, the system moves to the **Ask for More**.

Once sufficient input is available, the system proceeds to the **Predict Disease** activity. In this step, machine learning models analyze the processed symptoms and generate a probable diagnosis. The system then evaluates whether the prediction has high confidence and whether the condition is clearly identified.

If the prediction confidence is low or there is ambiguity, the system moves to **Ask for Clarification**, where it requests specific input or confirmation from the user. This ensures that the system improves accuracy by refining the input data.

If the condition is clearly identified, the system moves to the **Generate Advice** phase. This phase consists of multiple sub-activities:

- **Analyze Specific Disease Features:** The system evaluates disease-specific parameters and constraints.
- **Generate Specific Condition Advice:** Provides targeted recommendations such as treatment plans or precautions.
- **Generate General Self-Care Advice:** Offers general health suggestions for the user.

These activities work together to produce comprehensive health guidance.

After generating the advice, the system performs **Consolidate Result**, where all recommendations are combined into a final structured output. This ensures that the response is complete and easy to understand.

Finally, the system reaches the **Display Result** activity, where the generated output is presented to the user through the chatbot interface. The process then ends at the **final state**, represented by a double circle.

Workflow Summary

1. User enters symptoms
2. System processes input
3. Checks for sufficient data
4. Predicts disease using ML models
5. Requests clarification if needed

6. Generates advice (specific + general)
7. Consolidates results
8. Displays output to user

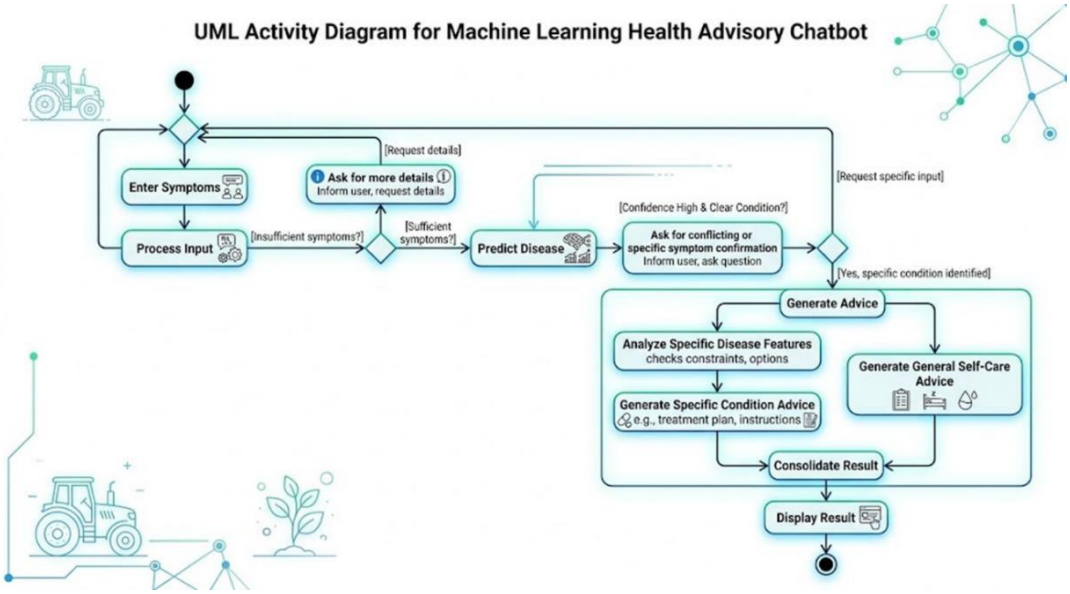

Fig 5.2.4: Activity diagram

5.2.5 Collaboration Diagram

A collaboration diagram, also known as a communication diagram, is a type of UML diagram that represents the interactions between various components or objects within a system. It focuses on how these components collaborate with each other to perform a specific task. Unlike sequence diagrams that emphasize the order of interactions over time, collaboration diagrams highlight the structural organization of objects and the flow of messages between them.

In the *Machine Learning Integrated Health Advisory Chatbot System*, the collaboration diagram illustrates how different modules such as **User Interface, Symptom Handler, Machine Learning Prediction Engine, Decision Support Module, and Response Generator** interact to process user input and generate appropriate health recommendations.

The process begins with the **User**, who interacts with the system by providing symptoms such as fever or cough. This interaction is handled by the **User Interface module**, which receives the input and displays system responses. The first message involves the user providing symptoms, which are then submitted as raw input to the system.

The **User Interface** forwards the input to the **Symptom Handler module**. The Symptom Handler processes the input by normalizing and extracting relevant symptom features. This includes cleaning the data, removing inconsistencies, and converting it into a structured format suitable for analysis. The processed data is then passed to the next module.

The **Machine Learning Prediction Engine** receives the processed data and performs the core computation. It executes machine learning models to predict the most probable disease. The module evaluates the model results and calculates confidence levels for predictions. After processing, it consolidates the results and forwards them to the Decision Support Module.

The **Decision Support Module** plays a crucial role in interpreting the prediction results. It applies predefined rules and combines prediction outputs with data from the medical knowledge base. It then generates a care plan that includes recommended medicines, precautions, and other health advice. The module ensures that the output is meaningful and actionable for the user.

Next, the **Response Generator module** takes the care plan and converts it into a user-friendly response. It processes the results, adds necessary context, and generates a readable output in natural language format. This ensures that the user can easily understand the system's recommendations.

Finally, the generated response is sent back to the **User Interface**, which displays the final result to the user. This completes the interaction cycle, ensuring a smooth flow of data and communication across all system components.

Workflow Summary

1. User enters symptoms through interface
2. Interface forwards input to Symptom Handler
3. Symptom Handler processes and structures data
4. ML Prediction Engine predicts disease and confidence

5. Decision Support Module generates care plan
6. Response Generator formats output
7. Final response is displayed to the user

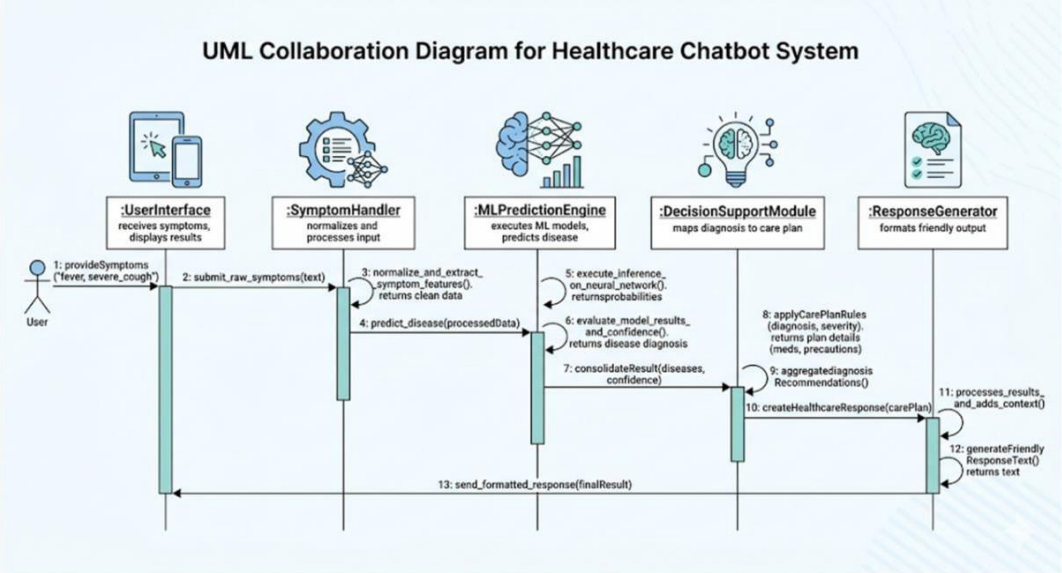

Fig 5.2.5: Collaboration diagram

5.2.6 State Diagram

A state machine diagram in Unified Modeling Language (UML) is used to represent the different states of a system and the transitions between those states based on events or conditions. It illustrates how the system behaves over time in response to external inputs and internal processes. The state diagram is particularly useful for understanding the dynamic behavior of the *Machine Learning Integrated Health Advisory Chatbot System*.

In this project, the state machine diagram describes the various stages through which the system passes, starting from idle state to generating and displaying the final response. Each state represents a specific phase of processing, and transitions indicate the movement from one state to another based on user actions or system events.

The process begins with the **Initial State**, represented by a filled circle, indicating the start of the system.

The system then moves into the **Idle State**, where it waits for user input. In this state, the system remains inactive until the user initiates a request by entering symptoms.

Once the user submits symptoms, the system transitions to the **Receiving Input State**. In this state, the chatbot interface receives the input data and displays the input screen. The system captures the symptoms provided by the user and prepares them for further processing.

After receiving the input, the system moves to the **Processing State**. In this stage, the system performs operations such as normalization and preprocessing of the raw input data. It extracts relevant symptom information and prepares the data in a structured format suitable for machine learning analysis.

The system then transitions to the **Predicting State**, which is the core computational stage. In this state, machine learning models are executed to analyze the processed symptoms and predict the most probable disease. The system performs inference and generates diagnosis results based on trained models.

Once the prediction is completed, the system moves to the **Generating Response State**. In this stage, the system applies decision-making rules and consolidates the prediction results with medical knowledge. It generates a user-friendly response that includes diagnosis, recommended medicines, and precautionary measures.

After generating the response, the system transitions to the **Displaying Output State**. In this state, the chatbot presents the final results to the user in a clear and understandable format. The system ensures that the output is well-structured and easy to interpret.

Finally, the system returns to the **Idle State**, indicating that the process is complete and the system is ready to accept new user input. This loop ensures continuous operation of the chatbot system.

Workflow Summary

1. System starts in Idle state
2. User submits symptoms
3. System receives input
4. Input is processed and normalized

5. ML models predict disease
6. Response is generated
7. Output is displayed to user
8. System returns to Idle state.

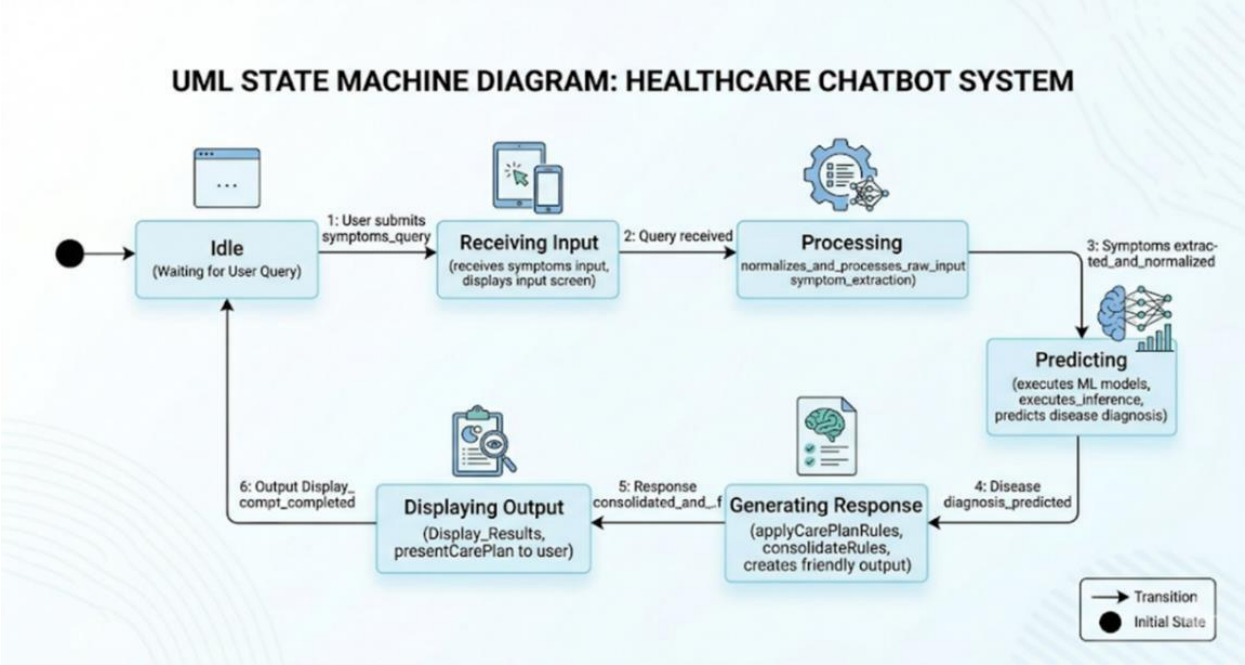

Fig 5.2.6: State Diagram

Chapter 6

SYSTEM IMPLEMENTATION

6.1 IMPLEMENTATION PROCESS

The implementation of the proposed *Machine Learning Integrated Health Advisory Chatbot System* is carried out through a structured sequence of phases, including data collection and preprocessing, model development and training, system integration, and deployment through a web/mobile-based chatbot interface for real-time health advisory services.

Data Collection and Preprocessing

The dataset used in this project consists of structured medical data containing symptoms and their corresponding diseases. This data is collected from publicly available healthcare datasets and online medical repositories. The dataset includes a wide range of symptoms such as fever, cough, headache, fatigue, nausea, and other health indicators associated with multiple diseases. To ensure better generalization, the dataset covers various disease categories and symptom combinations.

Before training the model, preprocessing techniques are applied to clean and structure the data. Missing values are handled, duplicate entries are removed, and categorical data is encoded into numerical format suitable for machine learning models. Feature selection techniques are applied to identify the most relevant symptoms for prediction. The data is then normalized to improve model performance and ensure consistency. The dataset is structured into input features (symptoms) and output labels (diseases), preparing it for model training.

Model Selection – XGBoost and Neural Networks

The system utilizes a combination of machine learning models, including XGBoost and Artificial Neural Networks, to achieve high accuracy in disease prediction. XGBoost is chosen for its efficiency, scalability, and strong performance on structured data, while Neural Networks are used to capture complex relationships between symptoms and diseases.

The models are trained using labeled datasets, where symptoms act as input features and diseases as target outputs. XGBoost provides fast and accurate predictions, whereas Neural

Networks enhance learning by identifying deeper patterns in the data. The combination of these models ensures robustness and improved prediction accuracy.

Training the Model

The dataset is divided into training, validation, and testing sets, typically in the ratio of 80:10:10. The models are trained using appropriate loss functions such as categorical cross-entropy and optimized using algorithms like Adam optimizer. During training, techniques such as cross-validation, regularization, and early stopping are applied to prevent overfitting and improve generalization.

Hyperparameter tuning is performed to optimize model performance. Parameters such as learning rate, number of estimators, and depth of trees (for XGBoost) or number of layers and neurons (for Neural Networks) are adjusted to achieve the best results. The training process is carried out using Python libraries such as Scikit-learn and TensorFlow.

Evaluation

After training, the models are evaluated using a separate test dataset. Performance metrics such as accuracy, precision, recall, and F1-score are used to assess the effectiveness of the model. Confusion matrix analysis is also performed to understand classification performance across different diseases.

The evaluation process ensures that the model provides reliable predictions and performs well on unseen data. Accuracy and loss graphs are analyzed to confirm proper convergence and stability of the model during training.

System Integration

The trained models are integrated into a web-based chatbot system using frameworks such as Flask or Django. The chatbot interface allows users to input their symptoms in a simple and interactive manner. The entered data is processed by the Symptom Handler module, which formats and prepares it for prediction.

The processed input is then passed to the prediction module, where the trained machine learning models generate disease predictions. The results are sent to the Decision Support module,

which maps predictions to appropriate medicines and precautionary measures using the medical knowledge base.

The Response Generator module converts the output into a user-friendly format and displays it through the chatbot interface. The system also stores user interactions and prediction results in a database for future analysis and improvement.

6.2 MODULE IMPLEMENTATION

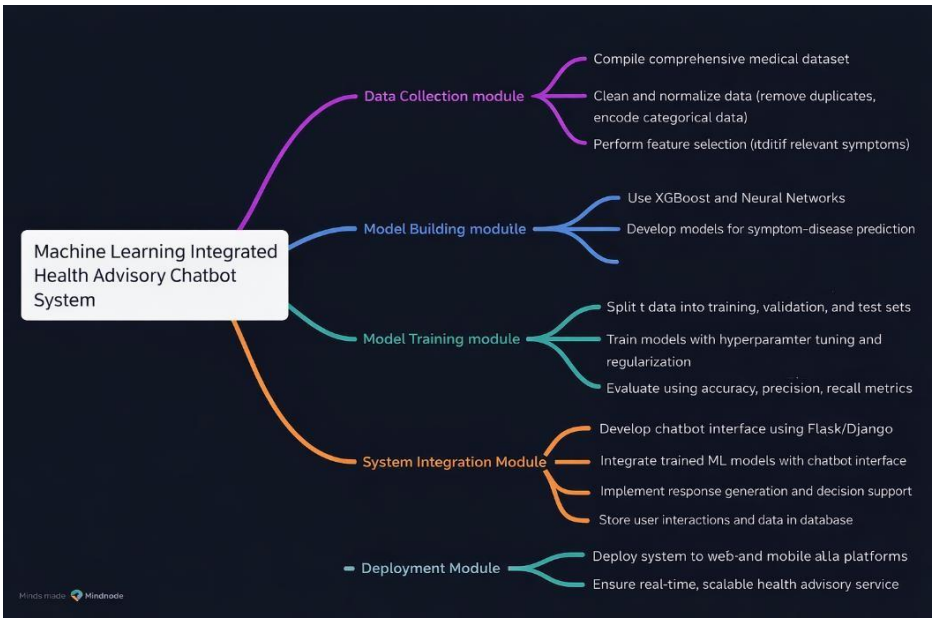

Fig 6.2.1 List of Modules

The *Machine Learning Integrated Health Advisory Chatbot System* consists of a series of interconnected modules that collaboratively perform disease prediction and provide health recommendations based on user-input symptoms. Each module is designed to execute a specific task, ensuring a modular, scalable, and efficient system architecture. This design supports real-time response, ease of integration with web and mobile platforms, and adaptability for future enhancements such as personalized healthcare and multilingual support. Each module is designed to execute a specific task, ensuring a modular, scalable, and efficient system architecture. Below is a detailed breakdown of each module:

- **Data collection module**

This module is responsible for compiling a comprehensive medical dataset that contains various symptoms and their corresponding diseases. The dataset includes a wide range of symptom combinations such as fever, cough, headache, fatigue, nausea, and other health indicators associated with multiple diseases. Data is collected from publicly available healthcare datasets, medical repositories, and online sources. The dataset is designed to cover diverse disease categories to improve the generalization capability of the model. Proper data collection ensures that the system can accurately learn patterns between symptoms and diseases, which is essential for reliable prediction.

- **Data preprocessing module**

In this stage, the collected data is cleaned and standardized to ensure consistency and quality. The preprocessing process includes removing duplicate records, handling missing values, and encoding categorical data into numerical format. Feature selection techniques are applied to identify the most relevant symptoms for disease prediction. The data is normalized to improve model performance and ensure uniformity. This module plays a crucial role in preparing the dataset for machine learning by transforming raw data into a structured and usable format.

- **Model building module (XGBoost & Neural Networks)**

This module focuses on selecting and building appropriate machine learning models for disease prediction. XGBoost is used for its high performance and efficiency in handling structured data, while Neural Networks are used to capture complex relationships between symptoms and diseases. The models are designed to learn patterns from the dataset and accurately map symptoms to diseases. Combining multiple models improves prediction accuracy and robustness, making the system more reliable.

- **Training module**

The training module is responsible for training the machine learning models using labeled data. The dataset is divided into training, validation, and testing sets to ensure proper evaluation. During training, the models learn to identify patterns and relationships between symptoms and diseases using supervised learning techniques. Hyperparameter tuning, regularization, and optimization

techniques such as the Adam optimizer are applied to enhance model performance and prevent overfitting. This module ensures that the models are well-trained and capable of making accurate predictions.

- **Evaluation module**

After training, the model performance is evaluated using various metrics such as accuracy, precision, recall, F1-score, and confusion matrix. These metrics help in assessing how well the model predicts diseases based on input symptoms. Visualization techniques such as accuracy and loss graphs are used to monitor training performance. This module ensures that the model meets the required performance standards before deployment.

- **Prediction module**

This module is responsible for generating disease predictions based on user input. When a user enters symptoms through the chatbot interface, the input is processed and passed to the trained machine learning models. The models analyze the symptoms and predict the most probable disease along with a confidence score. This module acts as the core functional component of the system.

- **Decision support module**

The decision support module interprets the prediction results and generates meaningful healthcare recommendations. It maps the predicted disease to appropriate medicines and precautionary measures using the medical knowledge base. This module ensures that users receive actionable advice rather than just prediction results, enhancing the usefulness of the system.

- **Response generation module**

This module converts the prediction and recommendation results into a user-friendly format. It generates clear and understandable responses that include disease details, suggested medicines, and precautionary steps. It may also provide follow-up suggestions or additional guidance. This improves the overall user experience and ensures effective communication.

- **User interface module**

The user interface module provides a chatbot-based interaction platform for users. It allows users to easily enter symptoms and receive results through a web or mobile application. The interface is

designed to be simple, intuitive, and user-friendly. It displays outputs such as predicted disease, medicines, and precautions in a structured format.

- **System integration module**

This module connects all components of the system, including data processing, machine learning models, decision support, and user interface. It handles backend processing, API communication, and data flow between modules. It also manages data storage and retrieval from the database. This ensures smooth and efficient operation of the entire system.

Chapter 7

SYSTEM STUDY AND TESTING

7.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase, and a business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis, the feasibility study of the proposed system is carried out. This is to ensure that the proposed system is not a burden to the users. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are:

1. Technical feasibility
2. Social feasibility
3. Economical feasibility

Technical feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands being placed on the user. The developed system must have modest requirements, as only minimal or no changes are needed for implementing this system. The proposed Machine Learning Integrated Health Advisory Chatbot System is developed using Python, machine learning libraries, and web technologies, which are lightweight and widely available, making it technically feasible**.**

Social feasibility

This aspect of the study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system; instead, they must accept it as a necessity. The level of acceptance by the users solely depends on the methods employed to educate them about the system and to make them familiar with it. Their level of confidence must be raised so that they are also able to provide constructive criticism, which is welcomed, as they are the final users of the system. The chatbot interface is designed to be simple and user-friendly, ensuring easy adoption.

Economical feasibility

This study is carried out to check the economic impact that the system will have. The amount of funds required for the development of the system is minimal. The expenditures must be justified. Thus, the developed system is well within the budget, as most of the technologies used such as Python, Scikit-learn, and TensorFlow are open-source and freely available. Only minimal costs are involved in deployment and hosting.

7.2 SOFTWARE TESTING

The purpose of testing is to discover errors. Testing is the process of trying to uncover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, modules, and the complete system. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

Types of Testing

There are many types of testing but the common and most efficient types of testing are:

1. Unit Testing
2. Integration Testing
3. System Testing
4. Acceptance Testing

Unit Testing

Unit testing is testing the smallest testable unit of an application. It is done during the coding phase by the developers. Unit Testing helps in finding bugs early in the development process. It is the testing of an individual unit or group of related units. It falls under the class of white box testing. It is often done by the programmer to test that the unit implemented is producing expected output against given input. Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid output.

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration.

Table 7.2.1: Unit testing

Test Case ID	**Input**	**Function**	**Expected Output**	**Result**
UT-01	Valid symptom input (fever, cough)	Predict Disease	Predicted disease	**Passed**
UT-02	Invalid/empty input	Validate Input	Error message / request for valid input	**Passed**
UT-03	Valid symptoms	Generate Advice	precautions displayed	**Passed**

Integration Testing

In Integration Testing different modules or components of a software application are tested as a combined entity. Integration tests are designed to test integrated software components to determine if they run as one system. Testing is more concerned with the interaction between modules.

In this project, the machine learning prediction module is integrated with the chatbot interface. Integration testing ensures that data flows smoothly from the user input in the frontend to the prediction engine in the backend and that the results are accurately displayed. It validates that the model receives the correct input format and returns predictions without errors. Integration testing ensures that data flows smoothly from the user input in the frontend to the prediction engine in the backend. It also ensures that the chatbot interface properly displays prediction results and recommendations.

Table 7.2.2: Integration testing

Test Case ID	Module Interaction	Input	Expected Output	Result
IT-01	UI → Prediction Module	Valid symptoms	Correct disease prediction	Passed
IT-02	Prediction → Decision Module	Predicted disease	Medicines and precautions generated	Passed
IT-03	Backend → UI	Prediction result	Output displayed correctly	Passed

System testing

System Testing evaluates the overall functionality and performance of the complete software. It ensures all modules are integrated correctly and the application satisfies user requirements. This testing simulates real-world scenarios to verify system behavior.

In this project, system testing confirms that symptom input, disease prediction, recommendation generation, and output display work together correctly. It ensures that the system provides accurate results, handles invalid inputs properly, and performs efficiently. It also checks system stability and responsiveness.

Table 7.2.3: System testing

Test Case ID	Scenario	Expected Output	Result
ST-01	Enter valid symptoms	Correct disease prediction	Passed
ST-02	Multiple users using system	System handles requests efficiently	Passed
ST-03	Invalid input	Error handling works correctly	Passed

Acceptance Testing

Acceptance Testing is a method of software testing where a system is tested for acceptability. The aim is to evaluate compliance with user requirements and it is a determine whether the system is ready for deployment.

In this project, acceptance testing is conducted with users to validate the chatbot functionality and usability. It ensures that the system meets user expectations and provides useful health advice. Feedback from users helps improve the system before final deployment.

Table 7.2.4: Acceptance testing

Test Case ID	Criteria	Expected Behavior	Result
AT-01	Disease Prediction	Correct disease displayed for given symptoms	Passed
AT-02	User Interface	Results displayed clearly and quickly	Passed
AT-03	System Response	Fast and accurate response generation	Passed
AT-04	Usability	Easy to use chatbot interface	Passed

Chapter 8

SOURCE CODE

Medicine_Recommendation.ipynb

```
import  pandas as pd
dataset = pd.read_csv(R'C:
\Users\User\Music\MEDICINE_RECOMMENDATION\DATASET\Training.csv')
dataset
dataset.shape
len(dataset['prognosis'].unique())
dataset['prognosis'].unique()
from sklearn.model_selection import train_test_split
X = dataset.drop('prognosis', axis=1)
y = dataset['prognosis']
X
# encoding prognonsis
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
le.fit(y)
Y = le.transform(y)
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3, random_state=20)
X_train.shape , y.train.shape
X_test.shape , y_test.shape
from xgboost import XGBClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
# Create an XGBoost Classifier model
```

```
xgb_model = XGBClassifier(max_depth=5, n_estimators=100, random_state=42)  # You can adjust max_depth and n_estimators as needed
# Train the XGBoost model
xgb_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = xgb_model.predict(X_test)

# Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Display confusion matrix
cm_display = ConfusionMatrixDisplay(confusion_matrix=cm)
cm_display.plot(cmap=plt.cm.Blues)
plt.title('XGBoost Confusion Matrix')
plt.show()

# Calculate and print accuracy
accuracy = xgb_model.score(X_test, y_test)
print(f"XGBoost Model Accuracy: {accuracy:.2f}")
from sklearn.metrics import confusion_matrix, classification_report
# Classification report
print(classification_report(y_test, y_pred))
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
```

```
# Create an XGBoost Classifier model
xgb_model = XGBClassifier(max_depth=5, n_estimators=100, random_state=42,
eval_metric="mlogloss") # Changed eval_metric to mlogloss for multi-class classification

# Train the model with evaluation metrics
eval_set = [(X_train, y_train), (X_test, y_test)]
xgb_model.fit(X_train, y_train, eval_set=eval_set, verbose=False)

# Extract accuracy metrics
results = xgb_model.evals_result()
train_accuracy = [1 - x for x in results['validation_0']['mlogloss']]  # Using mlogloss for multi-
class
test_accuracy = [1 - x for x in results['validation_1']['mlogloss']]  # Using mlogloss for multi-
class
# Plot accuracy curves
plt.figure(figsize=(8, 6))
plt.plot(train_accuracy, label="Training Accuracy")
plt.plot(test_accuracy, label="Testing Accuracy")
plt.xlabel("Boosting Rounds")
plt.ylabel("Accuracy")
plt.title("XGBoost Accuracy Curve")
plt.legend()
plt.grid()
plt.show()
# test 1:
print("predicted disease :",xgb_model.predict(X_test.iloc[0].values.reshape(1,-1)))
```

```
print("Actual Disease :", y_test[0])
sym_des = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/MEDICINE_RECOMMENDATION/DATASET/Symptom-severity.csv")
precautions = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/MEDICINE_RECOMMENDATION/DATASET/precautions_df.csv")
workout = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/MEDICINE_RECOMMENDATION/DATASET/workout_df.csv")
description = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/MEDICINE_RECOMMENDATION/DATASET/description.csv")
medications = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/MEDICINE_RECOMMENDATION/DATASET/medications.csv')
diets = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/MEDICINE_RECOMMENDATION/DATASET/diets.csv")
# custome and helping functions
#==========================helper funtions================
def helper(dis):
    desc = description[description['Disease'] == predicted_disease]['Description']
    desc = " ".join([w for w in desc])
    pre = precautions[precautions['Disease'] == dis][['Precaution_1', 'Precaution_2', 'Precaution_3', 'Precaution_4']]
    pre = [col for col in pre.values]
    med = medications[medications['Disease'] == dis]['Medication']
    med = [med for med in med.values]
    die = diets[diets['Disease'] == dis]['Diet']
    die = [die for die in die.values]
    wrkout = workout[workout['disease'] == dis] ['workout']
    return desc,pre,med,die,wrkout
```

symptoms_dict = {'itching': 0, 'skin_rash': 1, 'nodal_skin_eruptions': 2, 'continuous_sneezing': 3, 'shivering': 4, 'chills': 5, 'joint_pain': 6, 'stomach_pain': 7, 'acidity': 8, 'ulcers_on_tongue': 9, 'muscle_wasting': 10, 'vomiting': 11, 'burning_micturition': 12, 'spotting_ urination': 13, 'fatigue': 14, 'weight_gain': 15, 'anxiety': 16, 'cold_hands_and_feets': 17, 'mood_swings': 18, 'weight_loss': 19, 'restlessness': 20, 'lethargy': 21, 'patches_in_throat': 22, 'irregular_sugar_level': 23, 'cough': 24, 'high_fever': 25, 'sunken_eyes': 26, 'breathlessness': 27, 'sweating': 28, 'dehydration': 29, 'indigestion': 30, 'headache': 31, 'yellowish_skin': 32, 'dark_urine': 33, 'nausea': 34, 'loss_of_appetite': 35, 'pain_behind_the_eyes': 36, 'back_pain': 37, 'constipation': 38, 'abdominal_pain': 39, 'diarrhoea': 40, 'mild_fever': 41, 'yellow_urine': 42, 'yellowing_of_eyes': 43, 'acute_liver_failure': 44, 'fluid_overload': 45, 'swelling_of_stomach': 46, 'swelled_lymph_nodes': 47, 'malaise': 48, 'blurred_and_distorted_vision': 49, 'phlegm': 50, 'throat_irritation': 51, 'redness_of_eyes': 52, 'sinus_pressure': 53, 'runny_nose': 54, 'congestion': 55, 'chest_pain': 56, 'weakness_in_limbs': 57, 'fast_heart_rate': 58, 'pain_during_bowel_movements': 59, 'pain_in_anal_region': 60, 'bloody_stool': 61, 'irritation_in_anus': 62, 'neck_pain': 63, 'dizziness': 64, 'cramps': 65, 'bruising': 66, 'obesity': 67, 'swollen_legs': 68, 'swollen_blood_vessels': 69, 'puffy_face_and_eyes': 70, 'enlarged_thyroid': 71, 'brittle_nails': 72, 'swollen_extremeties': 73, 'excessive_hunger': 74, 'extra_marital_contacts': 75, 'drying_and_tingling_lips': 76, 'slurred_speech': 77, 'knee_pain': 78, 'hip_joint_pain': 79, 'muscle_weakness': 80, 'stiff_neck': 81, 'swelling_joints': 82, 'movement_stiffness': 83, 'spinning_movements': 84, 'loss_of_balance': 85, 'unsteadiness': 86, 'weakness_of_one_body_side': 87, 'loss_of_smell': 88, 'bladder_discomfort': 89, 'foul_smell_of urine': 90, 'continuous_feel_of_urine': 91, 'passage_of_gases': 92, 'internal_itching': 93, 'toxic_look_(typhos)': 94, 'depression': 95, 'irritability': 96, 'muscle_pain': 97, 'altered_sensorium': 98, 'red_spots_over_body': 99, 'belly_pain': 100, 'abnormal_menstruation': 101, 'dischromic _patches': 102, 'watering_from_eyes': 103, 'increased_appetite': 104, 'polyuria': 105, 'family_history': 106, 'mucoid_sputum': 107, 'rusty_sputum': 108, 'lack_of_concentration': 109, 'visual_disturbances': 110, 'receiving_blood_transfusion': 111, 'receiving_unsterile_injections': 112, 'coma': 113, 'stomach_bleeding': 114, 'distention_of_abdomen': 115, 'history_of_alcohol_consumption': 116, 'fluid_overload.1': 117, 'blood_in_sputum': 118, 'prominent_veins_on_calf': 119, 'palpitations':

```
120, 'painful_walking': 121, 'pus_filled_pimples': 122, 'blackheads': 123, 'scurring': 124, 'skin_peeling': 125, 'silver_like_dusting': 126, 'small_dents_in_nails': 127, 'inflammatory_nails': 128, 'blister': 129, 'red_sore_around_nose': 130, 'yellow_crust_ooze': 131}
diseases_list = {15: 'Fungal infection', 4: 'Allergy', 16: 'GERD', 9: 'Chronic cholestasis', 14: 'Drug Reaction', 33: 'Peptic ulcer diseae', 1: 'AIDS', 12: 'Diabetes ', 17: 'Gastroenteritis', 6: 'Bronchial Asthma', 23: 'Hypertension ', 30: 'Migraine', 7: 'Cervical spondylosis', 32: 'Paralysis (brain hemorrhage)', 28: 'Jaundice', 29: 'Malaria', 8: 'Chicken pox', 11: 'Dengue', 37: 'Typhoid', 40: 'hepatitis A', 19: 'Hepatitis B', 20: 'Hepatitis C', 21: 'Hepatitis D', 22: 'Hepatitis E', 3: 'Alcoholic hepatitis', 36: 'Tuberculosis', 10: 'Common Cold', 34: 'Pneumonia', 13: 'Dimorphic hemmorhoids(piles)', 18: 'Heart attack', 39: 'Varicose veins', 26: 'Hypothyroidism', 24: 'Hyperthyroidism', 25: 'Hypoglycemia', 31: 'Osteoarthristis', 5: 'Arthritis', 0: '(vertigo) Paroymsal  Positional Vertigo', 2: 'Acne', 38: 'Urinary tract infection', 35: 'Psoriasis', 27: 'Impetigo'}

# Model Prediction function
def get_predicted_value(patient_symptoms):
    input_vector = np.zeros(len(symptoms_dict))
    for item in patient_symptoms:
        input_vector[symptoms_dict[item]] = 1
    return diseases_list[xgb_model.predict([input_vector])[0]]
# Test 1
# Split the user's input into a list of symptoms (assuming they are comma-separated) # itching,skin_rash,nodal_skin_eruptions
symptoms = input("Enter your symptoms..... ")
user_symptoms = [s.strip() for s in symptoms.split(',')]
# Remove any extra characters, if any
user_symptoms = [symptom.strip("[]' ") for symptom in user_symptoms]predicted_disease = get_predicted_value(user_symptoms)
```

```
desc, pre, med, die, wrkout = helper(predicted_disease)


print("==================predicted disease============")
print(predicted_disease)
print("==================description==================")
print(desc)
print("==================precautions==================")
i = 1
for p_i in pre[0]:
    print(i, ": ", p_i)
    i += 1

print("==================medications==================")
for m_i in med:
    print(i, ": ", m_i)
    i += 1

print("==================workout==================")
for w_i in wrkout:
    print(i, ": ", w_i)
    i += 1

print("==================diets==================")
for d_i in die:
    print(i, ": ", d_i)
    i += 1
```

```
# Test 1
# Split the user's input into a list of symptoms (assuming they are comma-separated) #
yellow_crust_ooze,red_sore_around_nose,small_dents_in_nails,inflammatory_nails,blister
symptoms = input("Enter your symptoms. .... ")
user_symptoms = [s.strip() for s in symptoms.split(',')]
# Remove any extra characters, if any
user_symptoms = [symptom.strip("[]' ") for symptom in user_symptoms]
predicted_disease = get_predicted_value(user_symptoms)
desc, pre, med, die, wrkout = helper(predicted_disease)

print("=================predicted disease============")
print(predicted_disease)
print("=================description==================")
print(desc)
print("=================precautions==================")
i = 1
for p_i in pre[0]:
    print(i, ": ", p_i)
    i += 1

print("=================medications==================")
for m_i in med:
    print(i, ": ", m_i)
    i += 1

print("=================workout==================")
```

```
for w_i in wrkout:
    print(i, ": ", w_i)
    i += 1


print("=================diets=================")
for d_i in die:
    print(i, ": ", d_i)
    i += 1


from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Create a Neural Network model (MLPClassifier)
nn_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=1000, random_state=42)  # You can adjust hidden layers and iterations

# Train the Neural Network model
nn_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_nn = nn_model.predict(X_test)

# Generate confusion matrix
cm_nn = confusion_matrix(y_test, y_pred_nn)
```

```
# Display confusion matrix
cm_display_nn = ConfusionMatrixDisplay(confusion_matrix=cm_nn)
cm_display_nn.plot(cmap=plt.cm.Blues)
plt.title('Neural Network Confusion Matrix')
plt.show()

# Calculate and print accuracy
accuracy_nn = nn_model.score(X_test, y_test)
print(f"Neural Network Model Accuracy: {accuracy_nn:.2f}")

from sklearn.metrics import confusion_matrix, classification_report
# Classification report
print(classification_report(y_test, y_pred))
```

models.py

```
from django.db import models
import numpy as np
import pickle

# Load the trained ML model (replace with the actual model path)
model =
pickle.load(open(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\FRONTEND\xgboost.pkl", 'rb'))

# Function to predict disease based on symptoms
def predict_disease(symptom_list):
    """
```

```
    Predict the disease based on the given symptoms.

    Args:
        symptom_list (list): A list of symptom values (binary: 1 if present, 0 if not).

    Returns:
        str: Predicted disease name.
    """
    symptoms_array = np.array(symptom_list).reshape(1, -1)  # Reshape for model input
    predicted_disease = model.predict(symptoms_array)[0]  # Get the prediction
    return predicted_disease
```

views.py

```
# views.py
from django.shortcuts import render
import numpy as np
import xgboost as xgb
import pandas as pd
import pickle
# Load your model and data (ensure they are available for prediction)
xgb_model =
pickle.load(open(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\FRONTEND\xgboost.pkl", 'rb'))
#xgb_model.load_model(r'C:\Users\User\Music\MEDICINE_RECOMMENDATION\FRONTEND\xgboost.pkl')
# Load datasets for descriptions, precautions, medications, etc.
description =
pd.read_csv(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\DATASET\description.csv")
```

```
precautions = pd.read_csv(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\DATASET\precautions_df.csv")
medications = pd.read_csv(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\DATASET\medications.csv")
diets = pd.read_csv(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\DATASET\diets.csv")
workout = pd.read_csv(r"C:\Users\kondu\Music\MEDICINE_RECOMMENDATION\DATASET\workout_df.csv")
# Define symptoms and diseases dictionaries
symptoms_dict = {
    'itching': 0, 'skin_rash': 1, 'nodal_skin_eruptions': 2, 'continuous_sneezing': 3, 'shivering': 4, 'chills': 5,
    'joint_pain': 6, 'stomach_pain': 7, 'acidity': 8, 'ulcers_on_tongue': 9, 'muscle_wasting': 10, 'vomiting': 11,
    'burning_micturition': 12, 'spotting_urination': 13, 'fatigue': 14, 'weight_gain': 15, 'anxiety': 16, 'cold_hands_and_feets': 17,
    'mood_swings': 18, 'weight_loss': 19, 'restlessness': 20, 'lethargy': 21, 'patches_in_throat': 22, 'irregular_sugar_level': 23,
    'cough': 24, 'high_fever': 25, 'sunken_eyes': 26, 'breathlessness': 27, 'sweating': 28, 'dehydration': 29, 'indigestion': 30,
    'headache': 31, 'yellowish_skin': 32, 'dark_urine': 33, 'nausea': 34, 'loss_of_appetite': 35, 'pain_behind_the_eyes': 36,
    'back_pain': 37, 'constipation': 38, 'abdominal_pain': 39, 'diarrhoea': 40, 'mild_fever': 41, 'yellow_urine': 42,
```

'yellowing_of_eyes': 43, 'acute_liver_failure': 44, 'fluid_overload': 45, 'swelling_of_stomach': 46, 'swelled_lymph_nodes': 47,

'malaise': 48, 'blurred_and_distorted_vision': 49, 'phlegm': 50, 'throat_irritation': 51, 'redness_of_eyes': 52,

'sinus_pressure': 53, 'runny_nose': 54, 'congestion': 55, 'chest_pain': 56, 'weakness_in_limbs': 57, 'fast_heart_rate': 58,

'pain_during_bowel_movements': 59, 'pain_in_anal_region': 60, 'bloody_stool': 61, 'irritation_in_anus': 62, 'neck_pain': 63,

'dizziness': 64, 'cramps': 65, 'bruising': 66, 'obesity': 67, 'swollen_legs': 68, 'swollen_blood_vessels': 69, 'puffy_face_and_eyes': 70,

'enlarged_thyroid': 71, 'brittle_nails': 72, 'swollen_extremeties': 73, 'excessive_hunger': 74, 'extra_marital_contacts': 75,

'drying_and_tingling_lips': 76, 'slurred_speech': 77, 'knee_pain': 78, 'hip_joint_pain': 79, 'muscle_weakness': 80,

'stiff_neck': 81, 'swelling_joints': 82, 'movement_stiffness': 83, 'spinning_movements': 84, 'loss_of_balance': 85,

'unsteadiness': 86, 'weakness_of_one_body_side': 87, 'loss_of_smell': 88, 'bladder_discomfort': 89, 'foul_smell_of_urine': 90,

'continuous_feel_of_urine': 91, 'passage_of_gases': 92, 'internal_itching': 93, 'toxic_look_(typhos)': 94, 'depression': 95,

'irritability': 96, 'muscle_pain': 97, 'altered_sensorium': 98, 'red_spots_over_body': 99, 'belly_pain': 100,

'abnormal_menstruation': 101, 'dischromic_patches': 102, 'watering_from_eyes': 103, 'increased_appetite': 104,

'polyuria': 105, 'family_history': 106, 'mucoid_sputum': 107, 'rusty_sputum': 108, 'lack_of_concentration': 109,

'visual_disturbances': 110, 'receiving_blood_transfusion': 111, 'receiving_unsterile_injections': 112, 'coma': 113,

```
    'stomach_bleeding': 114, 'distention_of_abdomen': 115, 'history_of_alcohol_consumption':
116, 'fluid_overload.1': 117,
    'blood_in_sputum': 118, 'prominent_veins_on_calf': 119, 'palpitations': 120, 'painful_walking':
121, 'pus_filled_pimples': 122,
    'blackheads': 123, 'scurring': 124, 'skin_peeling': 125, 'silver_like_dusting': 126,
'small_dents_in_nails': 127,
    'inflammatory_nails': 128, 'blister': 129, 'red_sore_around_nose': 130, 'yellow_crust_ooze': 131
}
diseases_list = {
    15: 'Fungal infection', 4: 'Allergy', 16: 'GERD', 9: 'Chronic cholestasis', 14: 'Drug Reaction',
33: 'Peptic ulcer disease',
    1: 'AIDS', 12: 'Diabetes', 17: 'Gastroenteritis', 6: 'Bronchial Asthma', 23: 'Hypertension', 30:
'Migraine', 7: 'Cervical spondylosis',
    32: 'Paralysis (brain hemorrhage)', 28: 'Jaundice', 29: 'Malaria', 8: 'Chicken pox', 11: 'Dengue',
37: 'Typhoid', 40: 'Hepatitis A',
    19: 'Hepatitis B', 20: 'Hepatitis C', 21: 'Hepatitis D', 22: 'Hepatitis E', 3: 'Alcoholic hepatitis',
36: 'Tuberculosis',
    10: 'Common Cold', 34: 'Pneumonia', 13: 'Dimorphic hemorrhoids(piles)', 18: 'Heart attack',
39: 'Varicose veins',
    26: 'Hypothyroidism', 24: 'Hyperthyroidism', 25: 'Hypoglycemia', 31: 'Osteoarthritis', 5:
'Arthritis', 0: '(vertigo) Paroxysmal Positional Vertigo',
    2: 'Acne', 38: 'Urinary tract infection', 35: 'Psoriasis', 27: 'Impetigo'
}
# Helper function to retrieve disease information
def helper(dis):
    desc = description[description['Disease'] == dis]['Description'].values[0]
```

```
    pre = precautions[precautions['Disease'] == dis][['Precaution_1', 'Precaution_2', 'Precaution_3',
'Precaution_4']].values.tolist()[0]
    med = medications[medications['Disease'] == dis]['Medication'].values.tolist()
    die = diets[diets['Disease'] == dis]['Diet'].values.tolist()
    wrkout = workout[workout['disease'] == dis]['workout'].values.tolist()
    return desc, pre, med, die, wrkout

# Function to process input symptoms
def process_input(symptoms_input):
    symptoms_list = symptoms_input.split(",")
    input_vector = np.zeros(len(symptoms_dict))
    for symptom in symptoms_list:
        symptom = symptom.strip()  # Remove extra spaces if any
        if symptom in symptoms_dict:
            input_vector[symptoms_dict[symptom]] = 1
    return input_vector

# Model Prediction function
def get_predicted_value(patient_symptoms):
    input_vector = process_input(patient_symptoms)
    return diseases_list[xgb_model.predict([input_vector])[0]]

# Home page to input symptoms
def home(request):
    return render(request, 'index.html')
```

```
# Input page to enter symptoms
def input(request):
    return render(request, 'input.html')


# Output page to display predicted disease and details
def output(request):
    if request.method == 'POST':
        patient_symptoms = request.POST.get('symptoms', '')
        if patient_symptoms:
            predicted_disease = get_predicted_value(patient_symptoms)
            desc, pre, med, die, wrkout = helper(predicted_disease)
            return render(request, 'output.html', {
                'predicted_disease': predicted_disease,
                'description': desc,
                'precautions': pre,
                'medications': med,
                'diet': die,
                'workout': wrkout
            })
    return render(request, 'input.html')
```

Chapter 9

SCREEN LAYOUTS

9.1 INTERFACE LAYOUT:

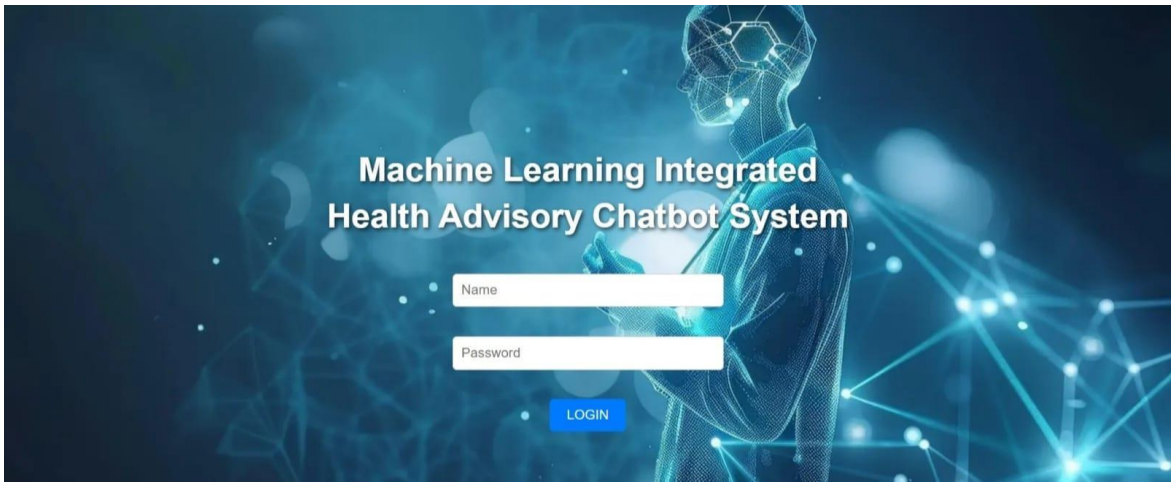

Fig 9.1 Interface layout

9.1.1 User Authentication Interface:

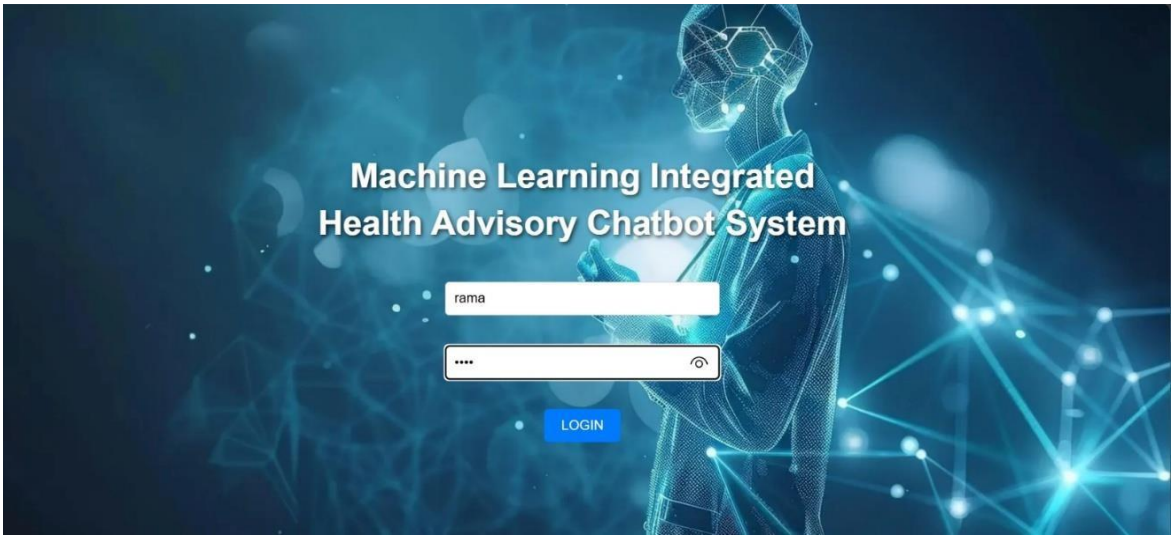

Fig 9.1.1 User Authentication Interface

9.1.2 Input Image:

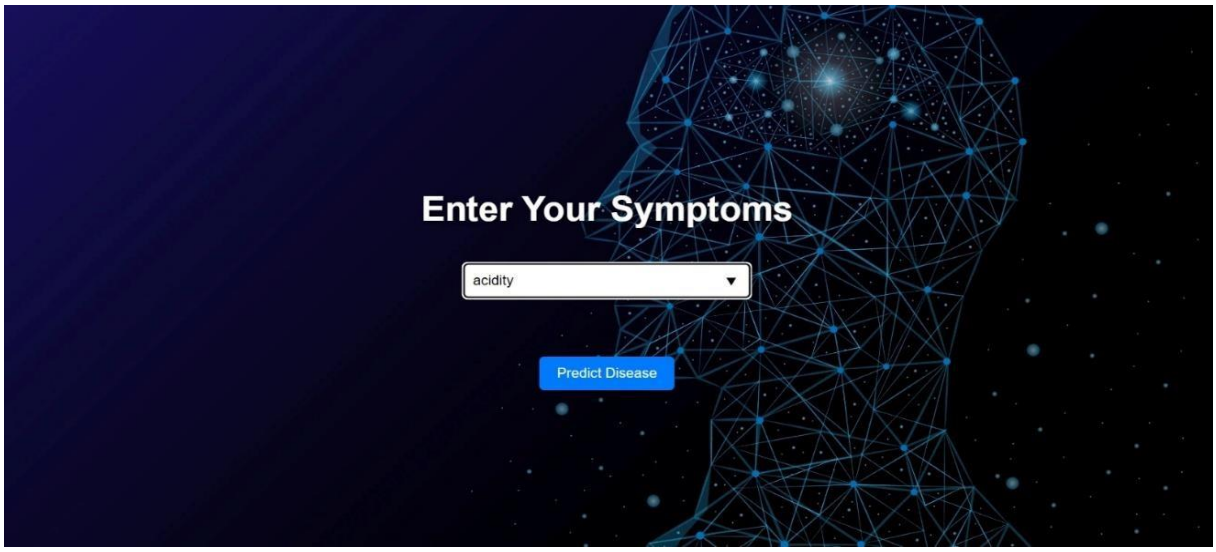

Fig 9.1.2 Input Image

9.2 RESULT:

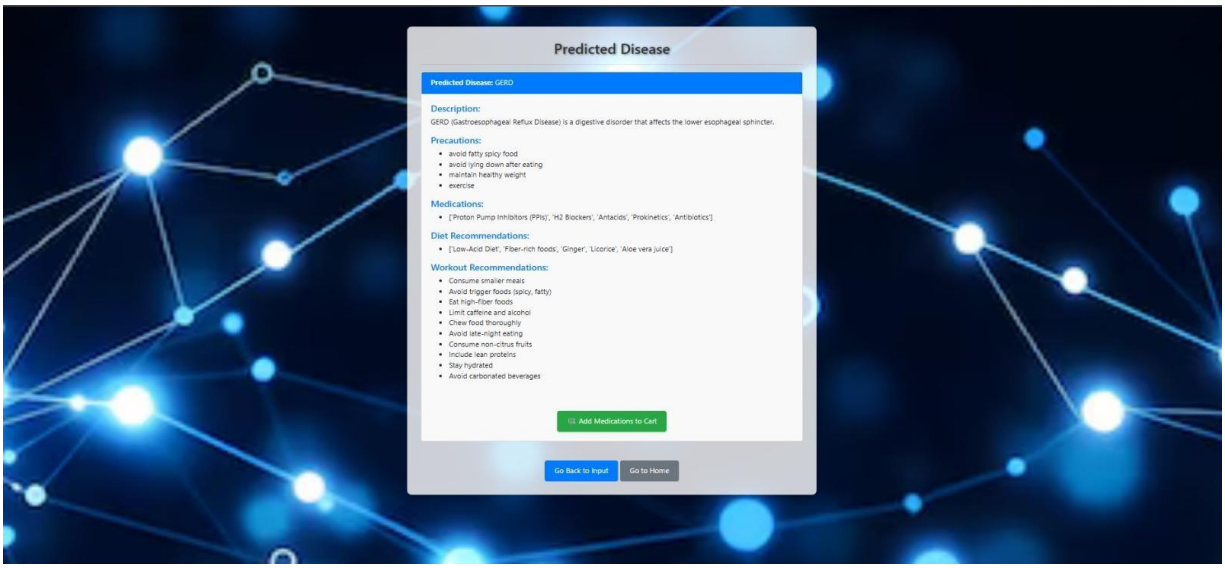

Fig 9.2. Output

Chapter 10

FUTURE ENHANCEMENTS

Future enhancements for the Machine Learning Integrated Health Advisory Chatbot System include incorporating advanced deep learning models to improve prediction accuracy and handle more complex symptom–disease relationships. The system can be extended to support real-time data integration from wearable devices such as smartwatches and fitness trackers, enabling continuous health monitoring and personalized recommendations. Additionally, integrating natural language understanding capabilities will allow users to interact with the chatbot using more conversational and flexible input formats.

A more comprehensive and dynamic medical knowledge base can be developed by incorporating real-time healthcare data and expert-verified medical guidelines. The system can also be enhanced to provide multi-language support, making it accessible to a wider range of users across different regions. Integration with telemedicine platforms will enable direct consultation with healthcare professionals when serious conditions are detected.

Furthermore, the implementation of personalized health profiling based on user history, lifestyle, and previous interactions will improve the relevance of recommendations. Advanced visualization techniques can be added to display health trends and risk analysis. A robust API can be developed to integrate the system with hospital management systems and healthcare applications.

Chapter 11

CONCLUSION

This project presents an intelligent, automated health advisory chatbot system using machine learning techniques, specifically XGBoost and Neural Networks, to provide accurate and real-time disease prediction based on user-entered symptoms. It addresses the limitations of traditional healthcare access by offering instant preliminary guidance without the need for immediate medical consultation. A structured medical dataset was utilized, with preprocessing techniques improving data quality and model performance. The machine learning models were trained and evaluated using various performance metrics to ensure reliability and accuracy. A user-friendly chatbot interface was developed using web technologies, enabling users to input symptoms and receive predictions along with recommended medicines and precautions. This system offers a scalable and efficient solution for improving healthcare accessibility and awareness, with significant potential for real-world adoption in digital healthcare platforms.

Chapter 12

BIBILOGRAPHY

1. T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
2. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
4. Esteva, A., Kuprel, B., Novoa, R. A., et al. (2017). Dermatologist-level classification of skin cancer with deep neural networks. Nature, 542(7639), 115–118.
5. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. New England Journal of Medicine, 380(14), 1347–1358.
6. Kourou, K., Exarchos, T. P., Exarchos, K. P., et al. (2015). Machine learning applications in cancer prognosis and prediction. Computational and Structural Biotechnology Journal, 13, 8–17.
7. Miotto, R., Li, L., Kidd, B. A., & Dudley, J. T. (2016). Deep patient: An unsupervised representation to predict the future of patients from electronic health records. Scientific Reports, 6, 26094.
8. Obermeyer, Z., & Emanuel, E. J. (2016). Predicting the future — Big data, machine learning, and clinical medicine. New England Journal of Medicine, 375(13), 1216–1219.
9. Topol, E. J. (2019). High-performance medicine: The convergence of human and artificial intelligence. Nature Medicine, 25(1), 44–56.
10. Deo, R. C. (2015). Machine learning in medicine. Circulation, 132(20), 1920–1930.
11. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
12. Abadi, M., Barham, P., Chen, J., et al. (2016). TensorFlow: A system for large-scale machine learning. USENIX Symposium on Operating Systems Design and Implementation (OSDI).